IPCA
School of Technology

Report on the Practical Project for Machine Learning Algorithms Course

Nuno Miguel Carvalho Araújo, No. 20078

Lecturer: Luís Ferreira

Barcelos, 2026

Contents

1 Introduction **1**

1.1 Contextualization . . . 1

1.2 Motivation and Objectives . . . 1

2 Problem Definition and Scope **2**

2.1 Market Context and Motivation . . . 2

2.2 Problem Formulation as Regression Task . . . 2

3 Data Comprehension **3**

3.1 Dataset Overview . . . 3

3.2 Target Variable Analysis . . . 3

3.2.1 Descriptive Statistics . . . 3

3.3 Feature Relationships with Price . . . 5

3.3.1 Correlation Analysis . . . 5

3.3.2 Property Characteristics . . . 6

3.3.3 Categorical Features . . . 6

4 Data Preparation **9**

4.1 Missing Values Analysis . . . 9

4.2 Removal of Irrelevant Features . . . 9

4.2.1 Identifiers and URLs . . . 9

4.2.2 Temporal and Source Metadata . . . 10

4.2.3 Host-Specific Information . . . 10

4.2.4 Review and Availability Metrics . . . 10

4.2.5 Text Descriptions and Redundant Geographic Data . . . 10

4.3 Removal of Inconsistent Records . . . 10

4.4 Outlier Treatment . . . 11

4.4.1 Target Variable Outlier Removal . . . 11

4.4.2 Feature Outlier Capping . . . 12

4.5 Missing Values Imputation . . . 12

4.5.1 Identification of Redundant Features . . . 12

4.5.2 Numeric Feature Imputation with Median . . . 13

4.5.3 Binary Categorical Feature Imputation with Mode . . . 13

4.5.4 Non-Binary Categorical Feature Imputation . . . 14

4.5.5 Imputation Verification . . . 14

4.6 Feature Engineering: Amenities Transformation . . . 14

4.6.1 Amenities Distribution Analysis . . . 15
4.6.2 Binary Amenity Feature Creation . . . 15
4.6.3 Amenities Count Feature . . . 16
4.6.4 Original Feature Removal . . . 16
4.7 Categorical Feature Encoding . . . 17
4.7.1 Property Type: Rare Category Grouping . . . 17
4.7.2 One-Hot Encoding: Room Type and Property Type . . . 18
4.7.3 Label Encoding: Neighbourhood . . . 18
4.8 Feature Selection: Correlation Analysis . . . 19
4.8.1 Correlation with Target Variable . . . 19
4.8.2 Low Correlation Feature Removal . . . 20
4.9 Data Shuffling and Train-Test Split . . . 20
4.9.1 Data Shuffling . . . 21
4.9.2 Train-Test Split Strategy . . . 21
4.10 Feature Scaling . . . 22
4.10.1 StandardScaler Application . . . 22
4.10.2 Selective Feature Scaling . . . 23
4.10.3 Preventing Data Leakage . . . 23
4.11 Data Preparation Summary . . . 23

5 Model Development **25**
5.1 Candidate Models and Rationale . . . 25
5.1.1 Linear Regression . . . 25
5.1.2 Random Forest Regressor . . . 25
5.2 Evaluation Metrics . . . 26
5.2.1 Root Mean Squared Error (RMSE) . . . 26
5.2.2 Mean Absolute Error (MAE) . . . 27
5.2.3 Coefficient of Determination (R^2) . . . 27
5.3 Baseline Model Training . . . 27
5.3.1 Linear Regression: Training and Evaluation . . . 27
5.3.2 Random Forest: Training and Evaluation . . . 28
5.4 Initial Model Comparison . . . 28
5.5 Hyperparameter Tuning: Random Forest Optimization . . . 29
5.5.1 Tuning Strategy and Parameter Grid . . . 29
5.5.2 Grid Search Execution . . . 30
5.5.3 Tuned Model Performance . . . 30
5.5.4 Overfitting Mitigation Analysis . . . 31

5.6 Final Model Selection 31

6 Model Export and Deployment **33**

6.1 Model Serialization and Export 33

6.1.1 Export Strategy and Components 33

6.1.2 Export Verification 33

6.1.3 Deployment Readiness 34

6.2 Deployment Demonstration 34

7 Conclusion **35**

List of Figures

1 Complete price distribution. 4
2 Price distribution up to the 99th percentile. 4
3 Correlation matrix heatmap - all numeric features. 5
4 Scatter plots - price vs accommodates, bedrooms, beds, and bathrooms. 6
5 Bar chart - median price by room type. 6
6 Horizontal bar chart - top 10 property types by median price. 7
7 Geographic distribution density map and scatter plot. 8
8 Top 10 features with the highest proportion of missing values. 9
9 Price distribution after outlier removal. 11
10 Distribution of the 20 most common amenities. 15
11 Relationship between total amenity count and rental price. 16
12 Features with highest absolute correlation with rental price. 19
13 Performance comparison between Linear Regression and Random Forest. 29
14 Performance comparison between default and tuned Random Forest configurations. 31

List of Tables

1 Price Distribution Statistics . . . 3
2 Median Price by Room Type . . . 6
3 Median Price by Dominant Property Type . . . 7
4 Median Price by Property Type (Top 10) . . . 7
5 Summary of Feature Outlier Capping Using IQR Method . . . 12
6 Correlation of Host Listing Features with Price . . . 12
7 Skewness of Numeric Features with Missing Values . . . 13
8 Summary of Missing Values Imputation Strategy . . . 14
9 Summary of Amenities Feature Engineering . . . 17
10 One-Hot Encoding Summary for Room Type and Property Type . . . 18
11 Summary of Correlation-Based Feature Selection . . . 20
12 Train-Test Split Summary and Target Variable Distribution . . . 22
13 Dataset Transformation Summary . . . 24
14 Linear Regression Baseline Performance . . . 27
15 Random Forest Baseline Performance (Default Parameters) . . . 28
16 Baseline Model Comparison: Linear Regression vs Random Forest . . . 28
17 Optimal Hyperparameters Identified by Grid Search . . . 30
18 Random Forest Performance: Default vs Tuned Configuration . . . 31
19 Final Model Comparison: Complete Performance Evaluation . . . 31

1 Introduction

1.1 Contextualization

This project, developed within the course unit of Machine Learning Algorithms, focuses on exploring good practices in the preparation, development, and application of intelligent models based on Machine Learning algorithms.

1.2 Motivation and Objectives

The aim of this project is to design and develop intelligent models capable of addressing real-world problems of moderate complexity in the domains of Classification or Regression. This involves identifying or creating appropriate datasets, performing rigorous data preparation, and applying effective Feature Engineering techniques. Following this preparation, one or more machine learning models will be selected based on well-defined success criteria, and their performance will be thoroughly evaluated. When necessary, improvement actions will be implemented to refine the chosen models. The use of pipelines will support automation and efficiency throughout the workflow, and model combination strategies will also be considered. Additionally, the project seeks to encourage the production of clean, well-documented source code and the development of a structured technical report.

The objectives of this project are as follows:

- Consolidate knowledge of Artificial Intelligence and Machine Learning algorithms.
- Analyze real-world problems and translate them into computational tasks.
- Develop programming skills in Python.
- Strengthen experience in designing and evaluating intelligent models.
- Deepen understanding of the content of the course unit.

Thus, this project aims not only to build and evaluate machine learning models for real-world scenarios, but also to promote a critical and well-founded understanding of the methodological, computational, and analytical processes involved, thereby consolidating essential skills in the field of Machine Learning.

2 Problem Definition and Scope

The problem under study consists of predicting short-term rental prices for Airbnb listings in Mexico City, Mexico. More specifically, the goal is to estimate the nightly rental price of a listing based on its observable characteristics, such as location, type of accommodation, and available amenities.

2.1 Market Context and Motivation

The short-term rental market in Mexico City has become increasingly competitive and dynamic, driven by the proliferation of platforms such as Airbnb. Prices fluctuate according to a variety of determinants, including neighborhood characteristics, accommodation type, and the specific amenities available. Accurately setting rental rates requires hosts to balance profitability with market competitiveness, while guests rely on clear and fair pricing for informed decision-making.

This study focuses exclusively on Mexico City, enabling an in-depth examination of local pricing patterns. By analyzing neighborhood-specific trends, accommodation distributions, and amenity offerings, the research aims to provide insights into the factors that influence pricing within this urban context.

The dataset employed originates from a publicly accessible Airbnb repository [1], offering structured details on listings across Mexico City, which serve as the basis for model training and evaluation.

2.2 Problem Formulation as Regression Task

The problem addressed in this project is formulated as a supervised machine learning regression task. Regression is appropriate because the target variable, namely the Airbnb rental price, is a continuous numerical value rather than a discrete class or category.

The model is trained using historical Airbnb listing data, where each instance consists of a set of input features (such as location, room type, and amenities) and a corresponding rental price. During training, the model learns a function that maps these input features to a real-valued output.

The expected outcome of the model is a numerical prediction representing the estimated rental price of an Airbnb listing given its characteristics. Model performance is assessed by comparing predicted prices with actual prices using standard regression evaluation metrics, with the objective of minimizing prediction error and achieving accurate and reliable price estimates.

3 Data Comprehension

This section presents a comprehensive exploration of the dataset, focusing on understanding its structure, statistical properties, and the relationships between features and the target variable. The analysis emphasizes the most relevant characteristics that influence rental pricing.

3.1 Dataset Overview

The dataset comprises 26,401 Airbnb listings in Mexico City with 79 features describing property characteristics, host information, location data, and pricing details. These features include 46 numeric and 33 categorical variables.

Initial examination revealed several data quality issues requiring preprocessing:

- The target variable `price` was stored as a string with currency symbols (e.g., "$1,200.00"), requiring conversion to numeric format.
- Multiple features contained significant proportions of missing values.
- The dataset included both numeric and categorical variables with varying scales and distributions.

As a preliminary step, the target feature `price` was converted to numeric format to facilitate subsequent analyses and data relationship exploration.

3.2 Target Variable Analysis

The target variable, nightly rental price, exhibits characteristics typical of real estate pricing data: right-skewed distribution with a long tail of high-value properties.

3.2.1 Descriptive Statistics

Table 1 summarizes the price distribution, revealing substantial variability across listings.

Table 1: Price Distribution Statistics

Statistic	**Value (USD)**
Mean	1,989.29
Median	1,039.00
Standard Deviation	18,500.68
Minimum	66.00
25th Percentile	630.00
50th Percentile	1,039.00
75th Percentile	1,647.00
Maximum	900,000.00

The substantial difference between mean and median indicates positive skewness, suggesting that while most listings have moderate prices, a subset of luxury properties significantly increases the average. This distribution pattern is consistent with urban short-term rental markets where premium properties command disproportionately higher rates.

Figures 1 and 2 present histogram representations of the rental prices.

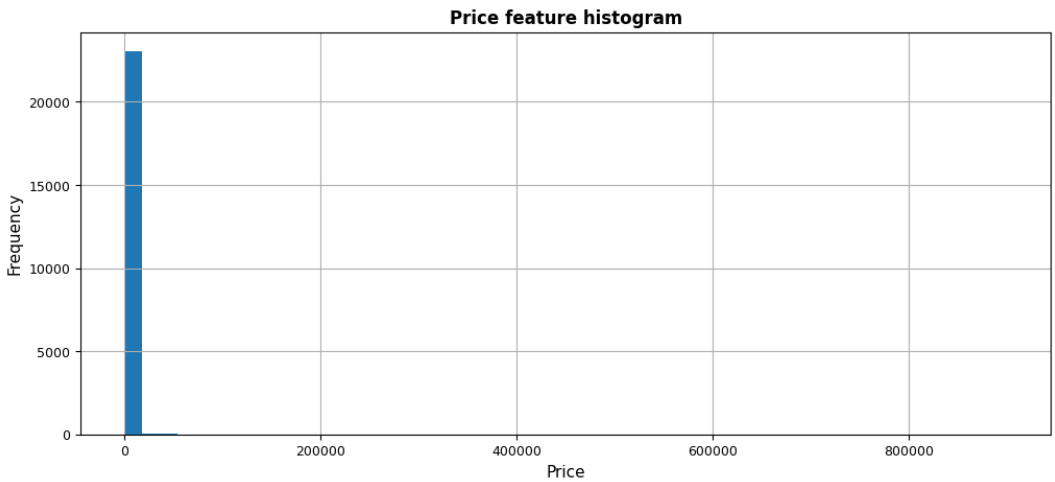

Figure 1: Complete price distribution.

Figure 1 displays the complete price range. However, extreme outliers extending beyond $10,000 per night severely compress the horizontal axis, obscuring the distribution of typical listings. To address this visualization limitation, Figure 2 excludes the top 1% of prices, revealing that most listings are concentrated below $5,000 per night with peak density in the lower price segments.

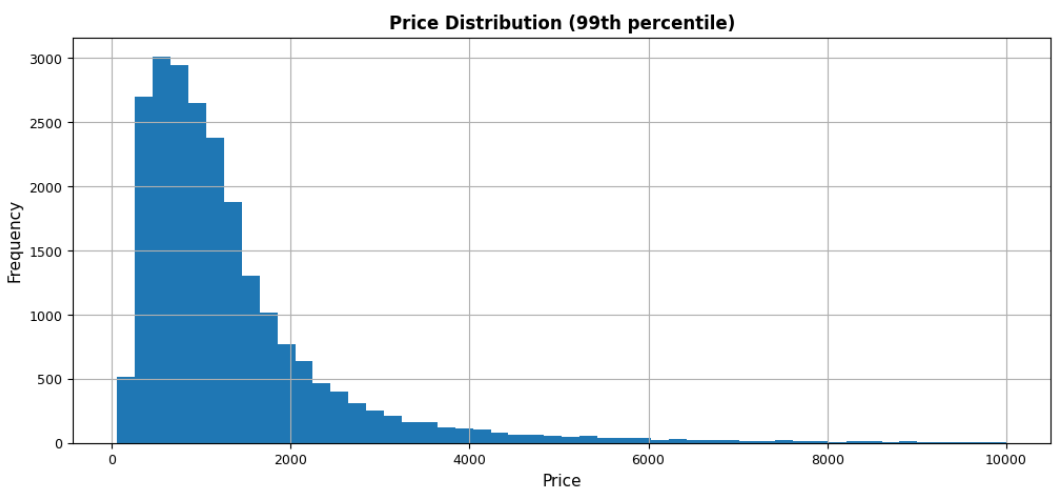

Figure 2: Price distribution up to the 99th percentile.

This right-skewed distribution confirms that extreme high-value properties constitute a small minority and indicates that outlier treatment will be necessary during data preparation.

3.3 Feature Relationships with Price

Understanding the relationship between property characteristics and rental prices is fundamental to effective feature engineering and model development.

3.3.1 Correlation Analysis

Initial correlation analysis across all numeric features revealed moderate linear relationships with price. Figure 3 presents the correlation matrix for numeric variables.

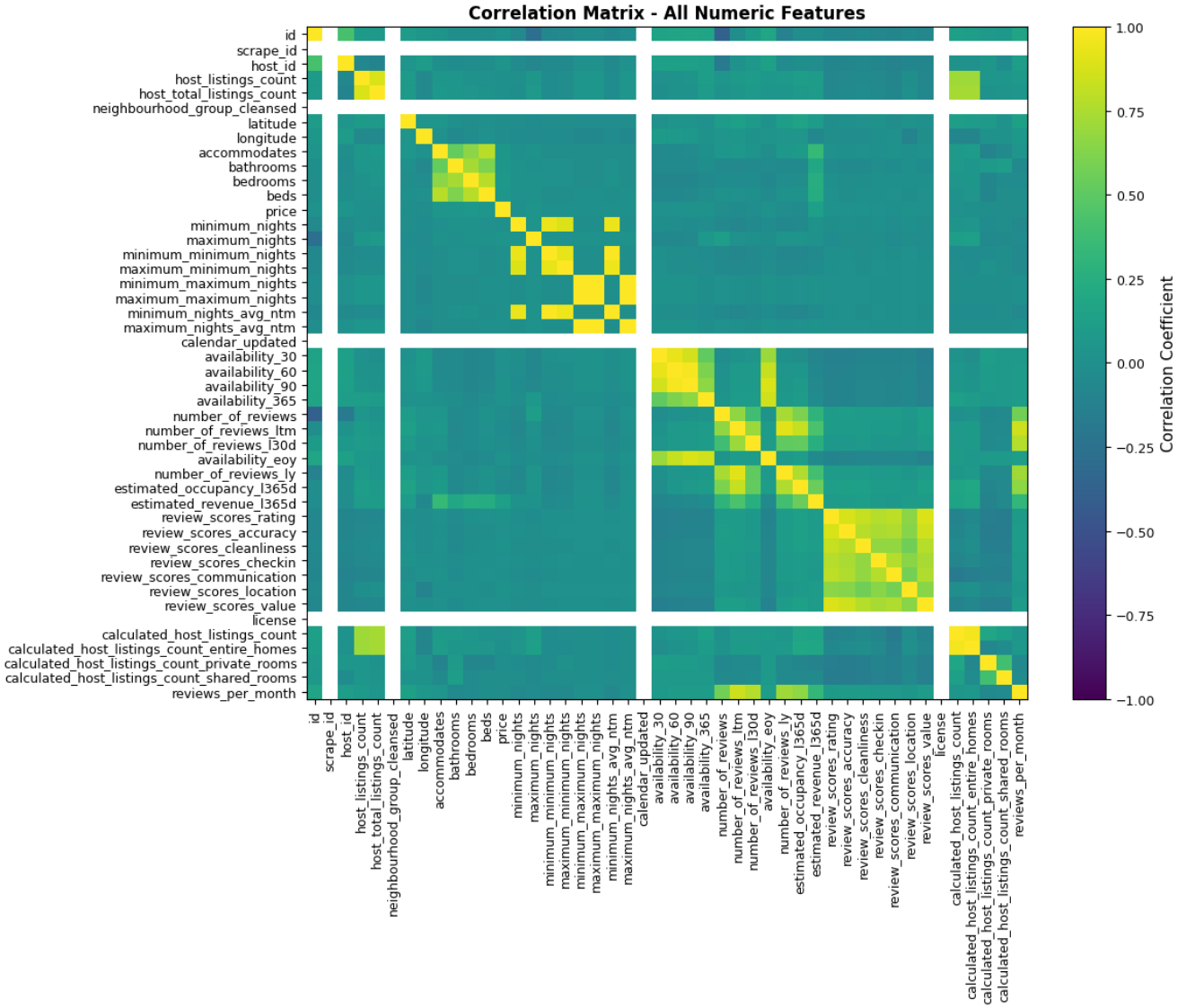

Figure 3: Correlation matrix heatmap - all numeric features.

While no single feature demonstrates exceptionally strong linear correlation with price, several property characteristics show meaningful associations worthy of further investigation.

3.3.2 Property Characteristics

Property capacity metrics (accommodates, bedrooms, beds, bathrooms) show positive associations with price, though Figure 4 reveals variance even among similar-capacity properties, suggesting that other factors (location, amenities, property type) play substantial roles in pricing.

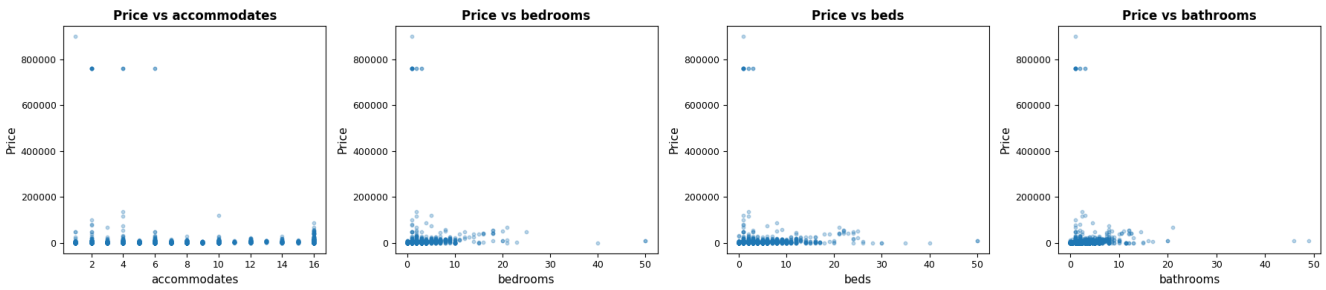

Figure 4: Scatter plots - price vs accommodates, bedrooms, beds, and bathrooms.

3.3.3 Categorical Features

Analysis of key categorical variables reveals distinct pricing patterns across accommodation types and locations.

Room Type: Table 2 shows median prices by room type, with hotel rooms highest, followed by entire homes/apartments, while shared shared rooms represent the most economical option.

Table 2: Median Price by Room Type

Room Type	Count	Median (USD)	Mean (USD)
Hotel room	63	1,994.00	88,009.73
Entire home/apt	15,570	1,276.00	1,888.26
Private room	7,289	563.00	1,508.55
Shared room	205	254.00	320.71

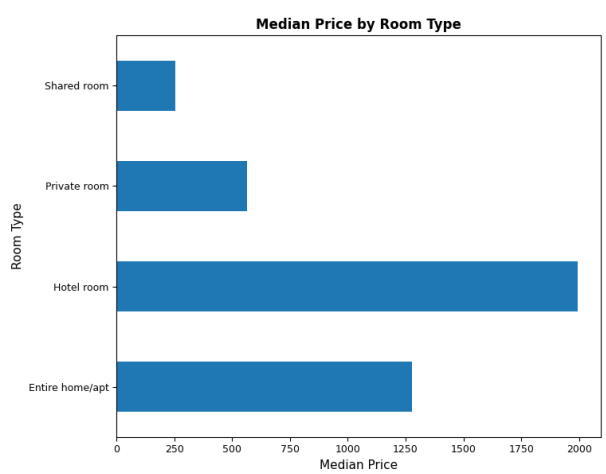

Figure 5: Bar chart - median price by room type.

Property Type: Table 3 shows "Entire rental unit" as the dominant property type in the dataset. Table 4 presents the top 10 property types by median price, revealing that unique accommodations (e.g., castles) command significantly higher prices than standard property types.

Table 3: Median Price by Dominant Property Type

Property Type	Count	Median (USD)	Mean (USD)
Entire rental unit	9,153	1,323.00	1,815.06

Table 4: Median Price by Property Type (Top 10)

Property Type	Count	Median (USD)	Mean (USD)
Castle	2	44,427.00	44,427.00
Entire villa	10	6,697.50	8,821.20
Farm stay	1	5,503.00	5,503.00
Entire home	688	2,860.00	5,624.33
Entire place	20	2,475.00	5,925.30
Entire home/apt	1	2,115.00	2,115.00
Private room in tent	1	2,037.00	2,037.00
Room in serviced apartment	13	1,990.00	2,509.46
Entire cottage	12	1,816.00	2,676.75
Entire chalet	3	1,695.00	1,350.00

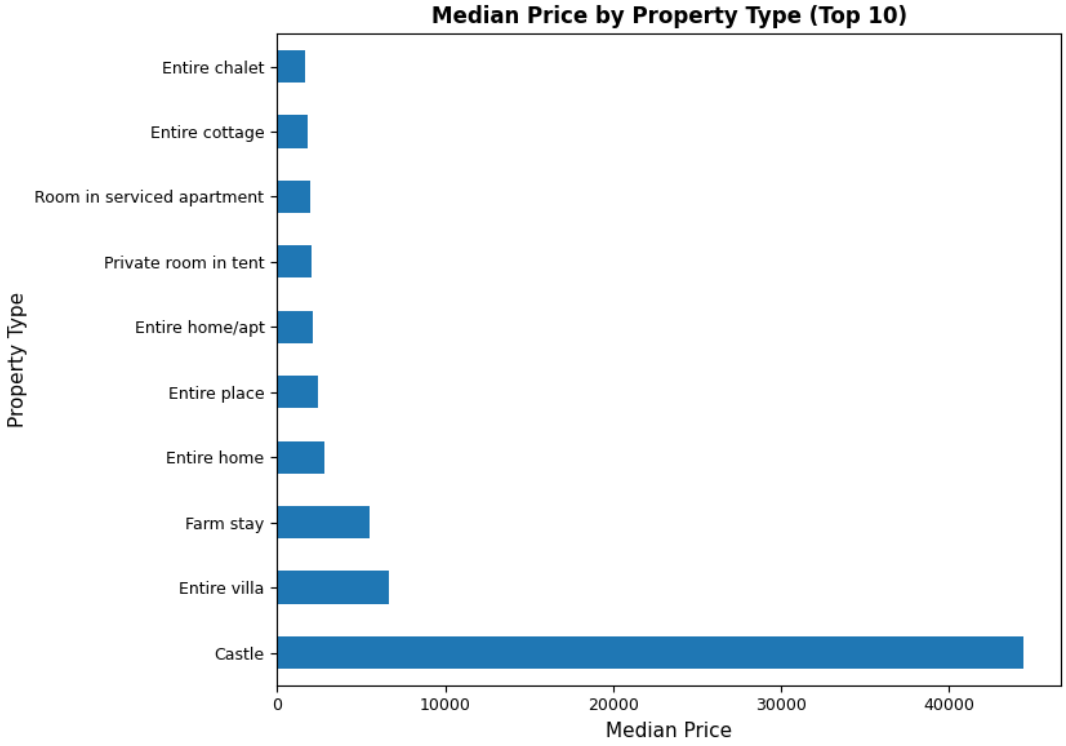

Figure 6: Horizontal bar chart - top 10 property types by median price.

Geographic Distribution: Listings are geographically concentrated in Mexico City's central areas (latitude: 19.20-19.50, longitude: -99.30 to -99.00). Figure 7 illustrates both listing density and price variation across the city, indicating that location significantly influences rental rates.

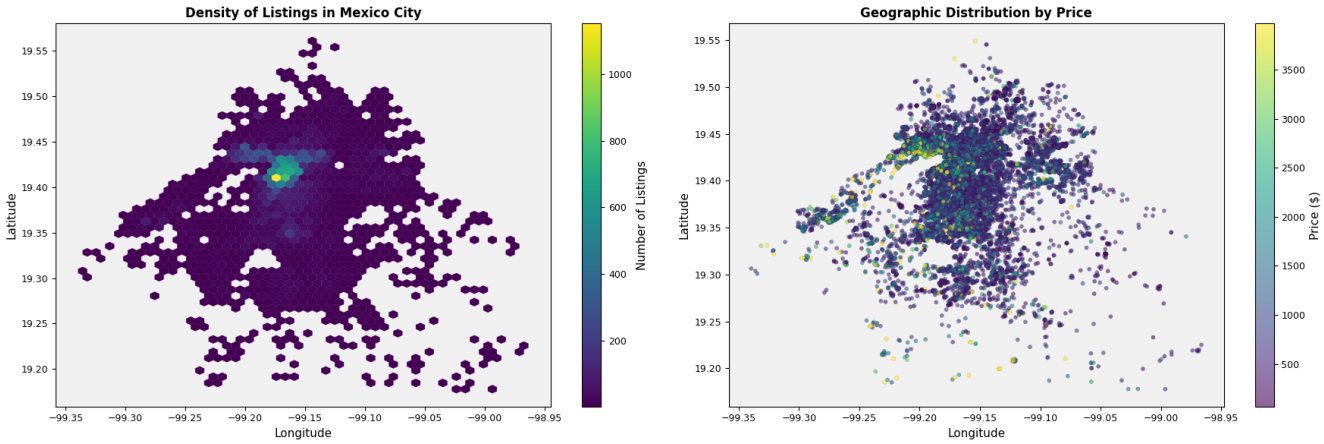

Figure 7: Geographic distribution density map and scatter plot.

4 Data Preparation

Data preparation constitutes a critical phase in the machine learning pipeline, directly impacting model performance and generalization capability. This section documents the systematic transformations applied to convert raw data into a clean and structured dataset suitable for model training.

4.1 Missing Values Analysis

Initial examination of the dataset revealed extensive missing data across multiple features. Figure 8 presents the top features with the highest proportion of missing values.

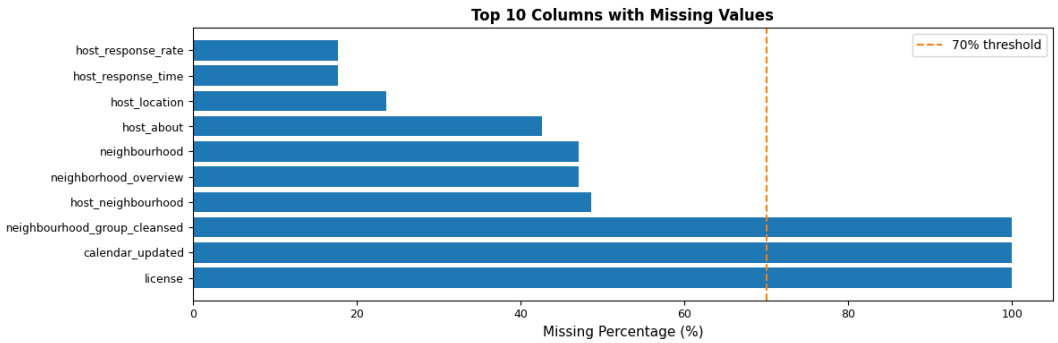

Figure 8: Top 10 features with the highest proportion of missing values.

Features with more than 70% missing values provide insufficient information for reliable model training and risk introducing noise rather than predictive signal. Such features were identified and removed from the dataset.

4.2 Removal of Irrelevant Features

Beyond missing data considerations, numerous features were identified as irrelevant to the price prediction task. These features were systematically removed based on their semantic content and relationship to the prediction objective. The removed features were categorized as follows:

4.2.1 Identifiers and URLs

Features such as `id`, `listing_url`, `picture_url`, `scrape_id`, and similar identifiers serve solely as unique keys or references to external resources. These provide no predictive information about rental prices and were removed to reduce dimensionality.

4.2.2 Temporal and Source Metadata

Features including `last_scraped`, `calendar_last_scraped`, `calendar_updated`, and `source` capture data collection metadata rather than property characteristics. Since the objective is to predict prices based on listing attributes at the time of publication, these features were excluded.

4.2.3 Host-Specific Information

A comprehensive set of host-related features (`host_id`, `host_name`, etc.) describe host profiles and behavior patterns. While host reputation may indirectly influence pricing, these features primarily reflect post-listing metrics and host management practices rather than intrinsic property characteristics. Their removal focuses the model on property attributes observable by potential guests during the booking decision.

4.2.4 Review and Availability Metrics

Features such as `number_of_reviews`, `review_scores_rating`, `first_review`, `last_review`, and availability indicators (`availability_30`, `availability_365`) represent outcomes or temporal states subsequent to listing publication. Including such features would introduce temporal leakage, as they reflect post-publication dynamics rather than the property characteristics that should determine initial pricing.

4.2.5 Text Descriptions and Redundant Geographic Data

Unstructured text fields (`name`, `description`, `neighborhood_overview`) would require natural language processing techniques beyond the scope of this project. Additionally, `latitude` and `longitude` were removed in favor of the structured `neighbourhood_cleansed` feature, which provides adequate geographic granularity without the complexity of coordinate-based modeling.

This systematic removal reduced the feature space from 79 to 20 features, eliminating noise and computational overhead while retaining all features with direct predictive relevance to rental pricing.

4.3 Removal of Inconsistent Records

Data quality extends beyond missing values to encompass logical consistency of recorded values. Records containing invalid or impossible values were identified and removed to prevent distortion of model training.

The following validation rules were applied:

- **Price validity:** Records with price values less than or equal to zero or null were removed, as negative or zero rental prices are logically inconsistent with commercial listings.

- **Accommodation capacity:** Listings with `accommodates` values less than or equal to zero were removed, as properties must accommodate at least one guest.
- **Property characteristics:** Records with negative values for `bedrooms`, `beds`, or `bathrooms` were removed, as these physical attributes cannot have negative quantities.

This validation process removed 3274 records, representing 12.40% of the original dataset. The removal of inconsistent records ensures that the model learns from valid data patterns rather than data entry errors or system anomalies.

4.4 Outlier Treatment

Outliers can substantially impact model performance, particularly for algorithms sensitive to extreme values. A two-stage outlier treatment strategy was implemented to balance information retention with model robustness.

4.4.1 Target Variable Outlier Removal

Extreme price outliers were addressed by removing listings exceeding the 99th percentile (>$10,000.00 per night). These ultra-luxury properties represent a distinct market segment with pricing dynamics fundamentally different from mainstream rental properties. Their inclusion would force the model to accommodate extreme ranges, potentially degrading predictive accuracy for typical listings that constitute the vast majority of the market.

This removal strategy differs from capping because these extreme cases do not represent measurement errors but rather a qualitatively different property category. The removal of 231 listings (1.00%) focuses the model on the mainstream rental market while maintaining sufficient data volume for training.

Figure 9 illustrates the price distribution after outlier removal.

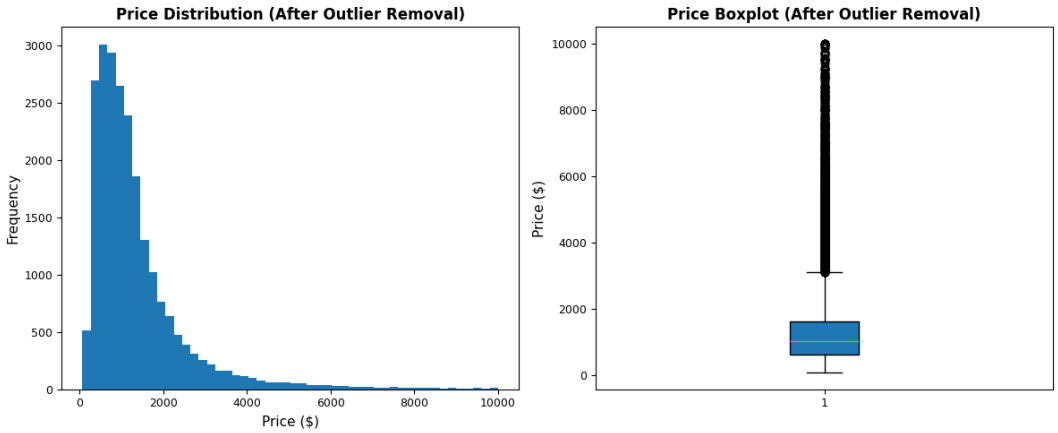

Figure 9: Price distribution after outlier removal.

4.4.2 Feature Outlier Capping

For key numeric features (`accommodates`, `bedrooms`, `beds`, `bathrooms`), outliers were capped rather than removed using the Interquartile Range (IQR) method. This approach defines outlier boundaries as Lower Bound $= Q_1 - 1.5 \times IQR$ and Upper Bound $= Q_3 + 1.5 \times IQR$, where Q_1 and Q_3 represent the first and third quartiles, and $IQR = Q_3 - Q_1$.

Values beyond these bounds were capped to the respective boundary values rather than removed. This technique preserves all records while limiting the influence of extreme values that may represent unusual but legitimate property configurations (e.g., properties accommodating 20 or more guests). Capping maintains the relative ordering of observations while preventing extreme values from dominating model parameter estimation.

Table 5 summarizes the outlier treatment for each feature.

Table 5: Summary of Feature Outlier Capping Using IQR Method

Feature	**Values Capped**	**Lower Bound**	**Upper Bound**
Accommodates	1028	-1.00	7.00
Bedrooms	583	-0.50	3.50
Beds	2136	-0.50	3.50
Bathrooms	480	-0.50	3.50

4.5 Missing Values Imputation

After removing features with excessive missing data and irrelevant columns, the remaining dataset still contained missing values requiring systematic treatment. The imputation strategy varied according to feature type and distributional characteristics.

4.5.1 Identification of Redundant Features

Before proceeding with imputation, correlation analysis revealed redundancy among host listing count features. Table 6 presents the correlation coefficients between these features and the target variable.

Table 6: Correlation of Host Listing Features with Price

Feature	**Correlation with Price**
calculated_host_listings_count_entire_homes	0.167
host_total_listings_count	0.137
calculated_host_listings_count	0.136
host_listings_count	0.098

The features `host_total_listings_count` (r = 0.137) and `calculated_host_listings_count` (r = 0.136) exhibited nearly identical correlations with price, differing by only 0.001. Furthermore,

`calculated_host_listings_count` represents an aggregate sum of property-type-specific subcategories (`entire_homes`, `private_rooms`, `shared_rooms`). Since the disaggregated features provide more granular information and `calculated_host_listings_count_entire_homes` demonstrated the highest correlation (r = 0.167), the three redundant aggregate features were removed while retaining the property-type-specific counts. This elimination of redundancy reduces multicollinearity risk and simplifies the feature space without information loss.

4.5.2 Numeric Feature Imputation with Median

Remaining numeric features with missing values were imputed using the median rather than the mean. This choice was motivated by the observed distributional characteristics of these features.

Table 7 presents the skewness coefficients for numeric features requiring imputation.

Table 7: Skewness of Numeric Features with Missing Values

Feature	**Skewness**
Bathrooms	1.57
Bedrooms	1.09
Beds	0.66

The features exhibited varying degrees of distributional skewness, with two features showing skewness values exceeding 1.0. The median provides a more robust measure of central tendency than the mean for skewed distributions, as it remains unaffected by extreme values in the distribution tails. Median imputation preserves the central location of the distribution while avoiding the inflation bias that mean imputation would introduce in right-skewed data.

4.5.3 Binary Categorical Feature Imputation with Mode

Binary categorical features encoded as 't' (true) or 'f' (false) were imputed using the mode (most frequent value). These features represent binary property characteristics (e.g., `host_is_superhost`, `instant_bookable`) where missing values likely indicate the absence of the characteristic or incomplete data entry.

Mode imputation for binary variables is appropriate because:

- It preserves the majority class distribution, maintaining the overall feature balance.
- Binary features lack intermediate values, making mode the only meaningful measure of central tendency.
- The imputation introduces minimal bias when the majority class is substantially larger, as the imputed values align with the most probable state.

Following imputation, these binary features were converted from string format ('t'/'f') to numeric format (1/0) to facilitate model training. This conversion represents True as 1 and False as 0, creating consistent numeric representations suitable for regression algorithms.

4.5.4 Non-Binary Categorical Feature Imputation

Remaining categorical features with missing values (non-binary) were imputed with the value 'Unknown'. This approach explicitly acknowledges the absence of information rather than artificially assigning the feature to an existing category. Creating an 'Unknown' category:

- Prevents distortion of existing category distributions.
- Allows the model to learn patterns specific to records with missing categorical information.
- Maintains data integrity by distinguishing genuine missing values from observed categories.

It should be noted that following the previous preprocessing steps, no categorical features with missing values remained that required this imputation strategy. Nevertheless, this treatment was retained in the preprocessing pipeline to ensure methodological completeness and applicability to scenarios where categorical missing values may persist at this stage.

4.5.5 Imputation Verification

After completing all imputation procedures, verification confirmed zero remaining missing values across the dataset. Table 8 summarizes the imputation strategy applied to each feature type.

Table 8: Summary of Missing Values Imputation Strategy

Feature Type	**Imputation Method**	**Features Affected**
Numeric (skewed)	Median	3
Binary Categorical	Mode	2
Non-Binary Categorical	'Unknown' category	0

4.6 Feature Engineering: Amenities Transformation

The `amenities` feature in the original dataset contained unstructured text representations of amenity lists (e.g., "["Wifi", "Kitchen", "Air conditioning"]"). While amenities significantly influence rental pricing, this raw text format is incompatible with machine learning algorithms. A systematic transformation process converted this unstructured data into structured numeric features.

4.6.1 Amenities Distribution Analysis

Initial analysis parsed all amenity lists to identify the frequency distribution of individual amenities across the dataset. Figure 10 presents the 20 most common amenities.

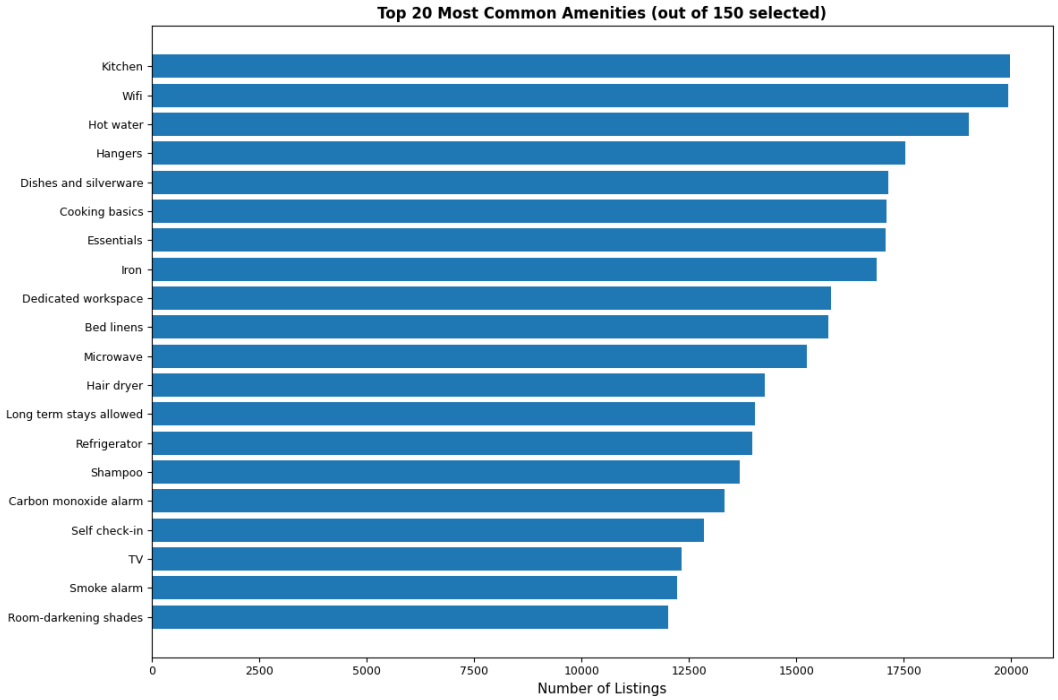

Figure 10: Distribution of the 20 most common amenities.

The distribution reveals that certain amenities (e.g., kitchen, WiFi, hot water) appear in the majority of listings, while others represent differentiating features for specific property types. This analysis informed the feature selection strategy for amenity encoding.

4.6.2 Binary Amenity Feature Creation

Rather than encoding all possible amenities (which would create excessive dimensionality with many sparse features), amenities appearing in at least 5% of listings were selected for binary encoding. This threshold balances:

- **Statistical reliability:** Features appearing in fewer than 5% of records provide limited training signal and risk overfitting to rare cases.
- **Dimensionality management:** Restricting to common amenities prevents feature space explosion while retaining the most impactful amenities.
- **Generalization capability:** Models trained on prevalent amenities can make reliable predictions for typical listings in the test set.

Each selected amenity was transformed into a binary feature (1 = present, 0 = absent) with the naming convention `has_[amenity_name]`. This encoding creates interpretable features where each coefficient directly represents the price impact of that specific amenity. The transformation generated 94 binary amenity features.

4.6.3 Amenities Count Feature

In addition to individual amenity indicators, an aggregate `amenities_count` feature was created, representing the total number of amenities per listing. This feature captures a complementary dimension of listing quality: while binary features indicate *which* amenities are present, the count feature reflects *how many* amenities are offered overall.

Figure 11 illustrates the relationship between total amenity count and rental price.

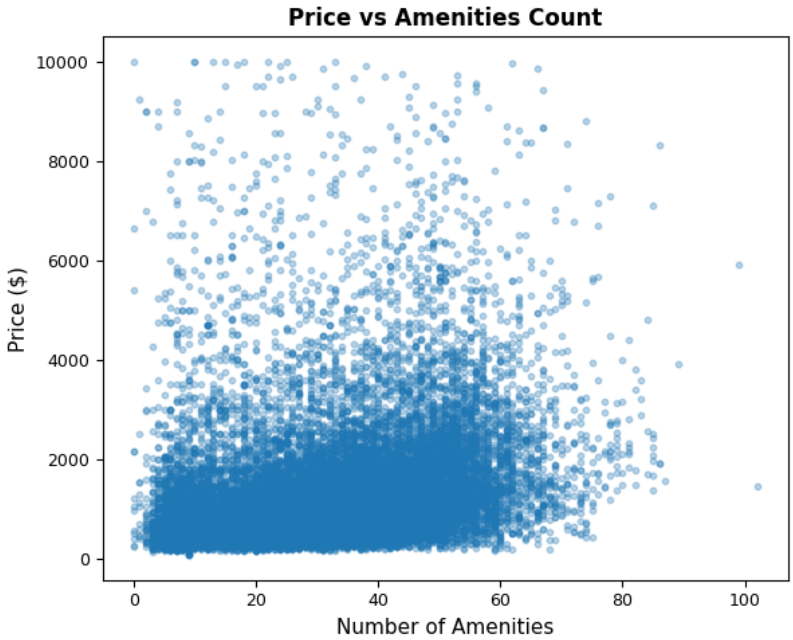

Figure 11: Relationship between total amenity count and rental price.

The positive relationship suggests that listings offering more amenities generally command higher prices, though substantial variance indicates that amenity quality and type matter beyond mere quantity. Both the count feature and individual amenity indicators provide complementary information: the count captures general listing completeness, while binary features enable the model to learn specific amenity value.

4.6.4 Original Feature Removal

After successful transformation, the original `amenities` text column was removed from the dataset. The structured binary and count features fully capture the amenity information in a

format suitable for model training, while eliminating the complexity of text processing during model deployment.

Table 9 summarizes the amenities transformation process.

Table 9: Summary of Amenities Feature Engineering

Transformation Step	**Result**
Total unique amenities identified	6362
Amenities meeting 5% threshold	94
Binary features created	94
Aggregate count feature created	1
Original text column removed	1

4.7 Categorical Feature Encoding

Categorical variables require transformation into numeric representations for compatibility with regression algorithms. The encoding strategy varied according to feature cardinality and semantic characteristics.

4.7.1 Property Type: Rare Category Grouping

The `property_type` feature exhibited high cardinality with 74 unique categories. However, many categories appeared infrequently, representing edge cases with insufficient data for reliable pattern learning.

To address this sparsity, property types appearing in less than 1% of listings were consolidated into an "Other" category. This threshold-based grouping:

- Reduces dimensionality by eliminating numerous sparse categories that would create mostly-zero columns after encoding.
- Prevents overfitting to rare property types that lack sufficient training examples.
- Retains all common property types that represent the mainstream rental market.
- Creates a meaningful "Other" category capturing unusual property types as a collective group.

After consolidation, the feature retained 12 distinct categories, balancing expressiveness with statistical reliability.

4.7.2 One-Hot Encoding: Room Type and Property Type

Both `room_type` and `property_type` (after rare category grouping) were encoded using one-hot encoding. This technique creates binary indicator variables for each category, where exactly one indicator equals 1 and all others equal 0 for each observation.

One-hot encoding was selected for these features because:

- **Nominal scale:** Both features represent categories without inherent ordering (e.g., "Entire home" is not inherently greater than "Private room").
- **Moderate cardinality:** After preprocessing, both features contain manageable numbers of categories that do not create excessive dimensionality.
- **Independence assumption:** Each category represents a distinct accommodation type with potentially unique pricing patterns that should be learned independently.

To avoid multicollinearity (the dummy variable trap), the first category was dropped from each encoded feature set (`drop_first=True`). This creates a reference category against which all other categories are compared, ensuring the feature matrix remains full rank and preventing linear dependence among predictors.

Table 10 summarizes the one-hot encoding results.

Table 10: One-Hot Encoding Summary for Room Type and Property Type

Original Feature	Unique Categories	Encoded Features Created
room_type	4	3
property_type (after grouping)	12	11

4.7.3 Label Encoding: Neighbourhood

The `neighbourhood_cleansed` feature contains 16 unique neighbourhoods, representing Mexico City's administrative divisions. Unlike `property_type`, one-hot encoding would create 15 binary features, substantially increasing dimensionality and risking sparsity issues.

Label encoding was applied instead, assigning each neighbourhood a unique integer identifier (0 to 15). While label encoding imposes artificial ordinality on nominal data, this approach was justified by:

- **Dimensionality reduction:** A single numeric feature replaces 15 potential binary columns, significantly reducing model complexity.
- **Tree-based model compatibility:** The selected Random Forest algorithm can effectively learn non-linear neighbourhood effects through splits on the encoded values, partially mitigating the ordinality limitation.

- **Information retention:** All neighbourhood distinctions are preserved, which means that no information is lost through aggregation.

The encoded feature `neighbourhood_encoded` ranges from 0 to 15, with each integer uniquely identifying a neighbourhood. While this encoding does not capture geographic relationships or ordinal structure, it provides sufficient information for the model to learn neighbourhood-specific pricing patterns.

The original categorical columns (`room_type`, `property_type`, `neighbourhood_cleansed`) were removed after encoding, leaving only their numeric representations in the dataset.

4.8 Feature Selection: Correlation Analysis

After encoding categorical variables, the dataset contained a substantial number of numeric features, including property characteristics, amenity indicators, and encoded categorical variables. Not all features contribute equally to price prediction, some exhibit minimal linear relationship with the target variable and may introduce noise rather than signal.

4.8.1 Correlation with Target Variable

Correlation analysis quantified the linear relationship between each numeric feature and rental price. Features were ranked by their Pearson correlation coefficient (r), which ranges from -1 (perfect negative correlation) to +1 (perfect positive correlation), with 0 indicating no linear relationship.

Figure 12 presents the features with the strongest positive and negative correlations with price.

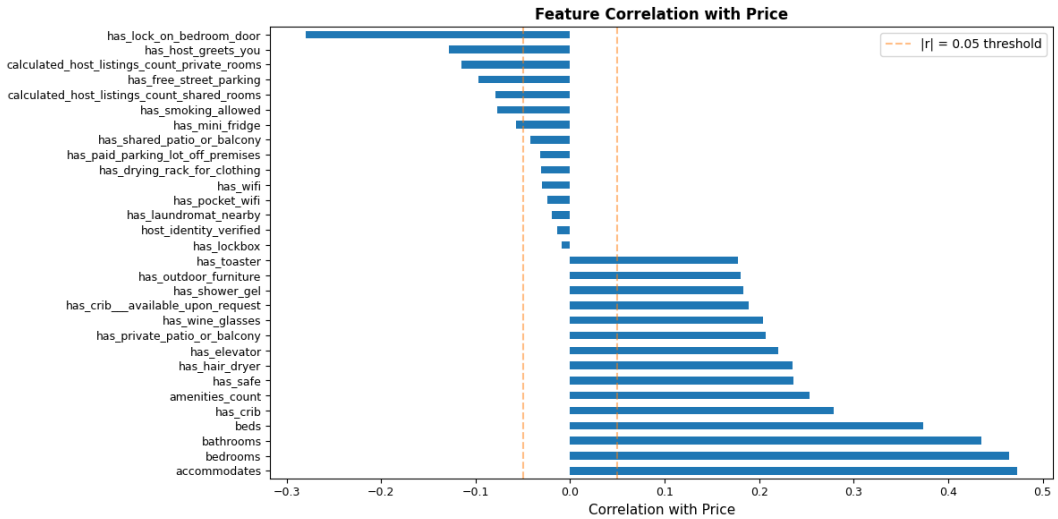

Figure 12: Features with highest absolute correlation with rental price.

The analysis revealed that property capacity features (`accommodates`, `bedrooms`, `bathrooms`, `beds`) and certain amenity indicators exhibited the strongest correlations. However, many features, particularly less common amenity indicators and some property type categories, demonstrated very weak linear relationships with price.

4.8.2 Low Correlation Feature Removal

Features with absolute correlation coefficients below 0.05 were removed from the dataset. This threshold was selected based on the following considerations:

- **Statistical significance:** Correlation coefficients below 0.05 explain less than 0.25% of price variance ($r^2 = 0.05^2 = 0.0025$), providing negligible predictive information.
- **Noise reduction:** Features with minimal correlation may capture random variation or idiosyncratic patterns that harm model generalization.
- **Dimensionality reduction:** Removing weak features simplifies the model, reduces computational cost, and decreases overfitting risk.
- **Focus on signal:** Retaining only features with meaningful correlations ensures the model learns from genuine pricing determinants rather than spurious associations.

It is important to note that correlation analysis captures only *linear* relationships. The subsequent Random Forest model can learn non-linear patterns among the retained features, potentially discovering interactions and complex relationships not evident in bivariate correlation analysis.

Table 11 summarizes the feature selection results.

Table 11: Summary of Correlation-Based Feature Selection

Selection Criterion	**Count**
Total numeric features before selection	105
Features with $\lvert r \rvert \geq 0.05$ (retained)	76
Features with $\lvert r \rvert < 0.05$ (removed)	29

The removal of 29 low-correlation features streamlined the dataset while retaining all features demonstrating meaningful linear associations with rental prices. This focused feature set provides a strong foundation for model training, emphasizing predictive signal over dataset comprehensiveness.

4.9 Data Shuffling and Train-Test Split

Proper dataset partitioning is essential to ensure unbiased model evaluation and prevent information leakage between training and testing phases.

4.9.1 Data Shuffling

Before partitioning, the dataset was randomly shuffled to eliminate any potential ordering effects present in the original data. The original dataset may contain implicit ordering based on collection timestamps, geographic sweeps, or other systematic patterns. Training on ordered data risks:

- **Temporal bias:** If data collection followed a temporal sequence, early/late observations may systematically differ.
- **Geographic clustering:** Sequential scraping of neighborhoods could create spatial autocorrelation in consecutive records.
- **Non-representative splits:** Without shuffling, systematic ordering could result in train and test sets with different underlying distributions.

Random shuffling (with `random_state=42` for reproducibility) ensures that both training and testing sets constitute representative samples of the overall population, supporting valid generalization assessment.

4.9.2 Train-Test Split Strategy

The shuffled dataset was partitioned into training (80%) and testing (20%) subsets. This allocation balances:

- **Training data volume:** 80% allocation provides 18,316 training samples, offering sufficient data for the model to learn complex pricing patterns.
- **Testing reliability:** 20% allocation yields 4,580 test samples, providing adequate sample size for stable performance estimation.
- **Standard practice:** The 80/20 split represents established convention in machine learning, facilitating comparison with other studies.

Critically, the split was performed *after* all data preparation steps (feature engineering, encoding, imputation) but *before* feature scaling. This sequence ensures:

- All preparation logic can be applied consistently to both sets.
- Scaling parameters are derived exclusively from training data, preventing information leakage.

Table 12 presents the split characteristics and confirms similar price distributions in both sets.

Table 12: Train-Test Split Summary and Target Variable Distribution

Characteristic	**Training Set**	**Testing Set**
Sample count	18,316 (80%)	4,580 (20%)
Feature count	93	93
Price Statistics		
Mean (USD)	1,356.07	1,364.33
Median (USD)	1,025.00	1,040.00

The similar price distributions across sets (mean difference ¡ 1%, median difference ¡ 2%) confirm that random shuffling produced balanced partitions, supporting valid model evaluation.

4.10 Feature Scaling

Machine learning algorithms perform more effectively when numeric features operate on comparable scales. Without scaling, features with large numeric ranges can dominate distance calculations and gradient computations, biasing model training.

4.10.1 StandardScaler Application

or key numeric features (`accommodates`, `bedrooms`, `beds`, `bathrooms`), outliers were capped rather than removed using the Interquartile Range (IQR) method. This approach defines outlier boundaries as Lower Bound $= Q_1 - 1.5 \times IQR$ and Upper Bound $= Q_3 + 1.5 \times IQR$, where Q_1 and Q_3 represent the first and third quartiles, and $IQR = Q_3 - Q_1$.

StandardScaler was applied to transform numeric features to zero mean and unit variance: $z = \frac{x-\mu}{\sigma}$ where x represents the original feature value, μ is the feature mean, and σ is the feature standard deviation.

This standardization:

- Centers all features around zero, facilitating interpretation of model coefficients.
- Equalizes feature scales, preventing large-magnitude features from dominating model training.
- Preserves distributional shape and relative distances between observations.
- Maintains interpretability, as standardized coefficients indicate effect size in standard deviation units.

4.10.2 Selective Feature Scaling

Importantly, only *continuous numeric features* were scaled. Binary features (amenity indicators, encoded categorical variables) were **not** scaled because:

- Binary features already operate on a consistent 0/1 scale requiring no adjustment.
- Scaling binary features to have mean 0 and variance 1 destroys their interpretable binary nature.
- Tree-based algorithms (including Random Forest) are invariant to monotonic transformations, making scaling of binary features unnecessary.

Features scaled included: `accommodates`, `bedrooms`, `beds`, `bathrooms`, `amenities_count`, and `neighbourhood_encoded`.

4.10.3 Preventing Data Leakage

Critical to proper scaling methodology, the scaler was *fit exclusively on training data* and then applied to both training and testing sets:

1. **Fit on training data:** μ and σ computed from training set only.
2. **Transform training data:** Training features scaled using training-derived parameters.
3. **Transform testing data:** Testing features scaled using the *same* training-derived parameters.

This procedure prevents data leakage, ensuring that no information from the testing set influences model training. If the scaler were fit on the entire dataset or on testing data, test set statistics would leak into the training process, artificially inflating performance estimates and invalidating model evaluation.

4.11 Data Preparation Summary

The data preparation phase transformed the raw dataset through systematic cleaning, feature engineering, and preprocessing operations. The original dataset of 20,778 listings with 75 features underwent multiple transformations to produce a refined dataset optimized for machine learning model training.

Table 13 presents a comparative overview of the dataset before and after preparation.

Table 13: Dataset Transformation Summary

Characteristic	**Before Preparation**	**After Preparation**
Total records	26,401	22,896
Total features	79	93
Missing values (columns)	43	0
Inconsistent records	3274	0
Categorical features	33	0
Target outliers (>99th percentile)	231	0
Final Splits		
Training samples	—	18,316 (80%)
Testing samples	—	4,580 (20%)

The final prepared dataset exhibits the following characteristics:

- **Complete data:** Zero missing values across all features.
- **Numeric representation:** All features are numeric, with categorical variables properly encoded.
- **Scaled features:** Continuous numeric features standardized to zero mean and unit variance.
- **Focused feature set:** Dimensionality reduced through removal of irrelevant, redundant, and low-correlation features.
- **Quality-controlled:** Inconsistent records removed and outliers appropriately treated.
- **Ready for training:** Data properly partitioned and prepared for model development without risk of information leakage.

The prepared dataset provides a clean, structured foundation for training and evaluating machine learning models for Airbnb price prediction.

5 Model Development

This section presents the model training, evaluation, and optimization process for predicting Airbnb rental prices. The development follows a systematic approach: initial training with default hyperparameters establishes baseline performance, comparative analysis identifies the most promising algorithm, and hyperparameter tuning optimizes the selected model. Throughout this process, rigorous evaluation on separate training and testing sets ensures reliable assessment of model generalization.

5.1 Candidate Models and Rationale

Numerous regression algorithms exist for price prediction tasks, including Support Vector Regression, Gradient Boosting, XGBoost, Neural Networks, and others. Each algorithm presents distinct advantages in terms of accuracy, interpretability, computational efficiency, and complexity.

For this academic project, two algorithms were strategically selected to balance pedagogical objectives with predictive performance, Linear Regression and Random Forest Regressor.

5.1.1 Linear Regression

Linear Regression serves as the baseline model, assuming a linear relationship between features and rental price. The model estimates coefficients β by minimizing the sum of squared residuals: $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$ where $\hat{y}$ represents the predicted price, β_0 is the intercept, and β_i are feature coefficients.

Linear Regression was selected primarily for its pedagogical value:

- **Conceptual simplicity:** The algorithm's straightforward mathematical foundation facilitates understanding of fundamental regression principles.
- **Interpretability:** Coefficients directly quantify each feature's contribution to price, providing clear insights into pricing determinants.
- **Baseline establishment:** Performance establishes a reference point for evaluating more sophisticated algorithms.
- **Computational efficiency:** Minimal training time enables rapid experimentation.

5.1.2 Random Forest Regressor

Random Forest is an ensemble learning method that constructs multiple decision trees during training and outputs the mean prediction across trees. Each tree is trained on a bootstrap sample

of the data, and splits consider random subsets of features, introducing diversity that reduces overfitting.

Random Forest was selected as the primary modeling approach due to its technical suitability for this prediction task:

- **Non-linear modeling:** Captures complex, non-linear relationships and feature interactions that Linear Regression cannot represent.
- **Robustness:** Ensemble averaging reduces variance and sensitivity to individual outliers or anomalous training examples.
- **Feature importance:** Provides interpretable feature importance scores, indicating which variables most influence predictions.
- **Minimal preprocessing requirements:** Handles mixed feature types and does not require strict distributional assumptions.

Given the distinct purposes of each model, Linear Regression for conceptual clarity and Random Forest for predictive accuracy, hyperparameter tuning was applied exclusively to the Random Forest model. This decision reflects that:

- Linear Regression has no critical hyperparameters requiring tuning.
- Random Forest justifies optimization effort to achieve maximum predictive accuracy.
- Focusing tuning resources on a single model enables more thorough exploration of the hyperparameter space.

5.2 Evaluation Metrics

Model performance is assessed using three standard regression metrics, each capturing different aspects of prediction quality.

5.2.1 Root Mean Squared Error (RMSE)

RMSE measures the average magnitude of prediction errors in the same units as the target variable: $RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$ where n is the number of observations, y_i is the actual price, and $\hat{y}_i$ is the predicted price. RMSE penalizes large errors more heavily than small errors due to the squaring operation, making it sensitive to outliers.

5.2.2 Mean Absolute Error (MAE)

MAE represents the average absolute prediction error: $MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$ and provides a more intuitive interpretation than RMSE, directly representing the average dollar amount by which predictions deviate from actual prices. Unlike RMSE, MAE treats all errors equally regardless of magnitude.

5.2.3 Coefficient of Determination (R^2)

R^2 quantifies the proportion of target variable variance explained by the model: $R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}{\sum_{i=1}^{n}(y_i-\bar{y})^2}$ where $\bar{y}$ is the mean of actual prices. R^2 ranges from 0 to 1, with higher values indicating better explanatory power. Unlike RMSE and MAE, R^2 is scale-independent, facilitating comparison across different datasets.

Together, these metrics provide comprehensive evaluation. RMSE and MAE quantify prediction accuracy in absolute terms, while R^2 indicates the model's explanatory power relative to a naive mean-based baseline.

5.3 Baseline Model Training

Both candidate models were initially trained using scikit-learn's default hyperparameters to establish baseline performance before optimization. This approach provides unbiased comparison of algorithms and identifies whether sophisticated tuning is necessary.

5.3.1 Linear Regression: Training and Evaluation

Linear Regression was trained using Ordinary Least Squares (OLS), fitting coefficients to minimize the sum of squared residuals on the training set. Training completed in 0.30 seconds, demonstrating the algorithm's computational efficiency.

Table 14 presents the Linear Regression performance on both training and testing sets.

Table 14: Linear Regression Baseline Performance

Metric	**Training Set**	**Testing Set**
RMSE (USD)	880.60	917.55
MAE (USD)	532.59	545.04
R^2 Score	0.4511	0.4456

The similar performance across training and testing sets (R^2 difference: 0.0055) indicates that Linear Regression does not overfit the training data. However, the moderate R^2 score suggests

limited capacity to capture the complex pricing patterns present in the dataset. The assumption of linear relationships appears insufficient to model the non-linear interactions and feature dependencies that characterize Airbnb pricing dynamics.

5.3.2 Random Forest: Training and Evaluation

Random Forest was trained with default parameters: 100 trees, unlimited depth, and minimum two samples per split. Training utilized all available CPU cores and completed in 11.02 seconds.

Table 15 presents the Random Forest baseline performance.

Table 15: Random Forest Baseline Performance (Default Parameters)

Metric	**Training Set**	**Testing Set**
RMSE (USD)	299.81	785.78
MAE (USD)	163.14	438.92
R^2 Score	0.9364	0.5934

Random Forest demonstrates substantially higher training performance compared to testing performance (R^2 difference: 0.3430). This gap indicates overfitting: the unrestricted tree depth allows individual trees to memorize training patterns, including noise and outliers, resulting in degraded generalization. Despite this overfitting tendency, Random Forest's test set performance significantly exceeds Linear Regression's performance, confirming the value of non-linear modeling for this task.

5.4 Initial Model Comparison

Table 16 compares both models across all evaluation metrics.

Table 16: Baseline Model Comparison: Linear Regression vs Random Forest

Model	**Test R^2**	**Test RMSE (USD)**	**Test MAE (USD)**
Linear Regression	0.45	917.55	545.04
Random Forest (Default)	0.59	785.78	438.92
Improvement	**31.11%**	**14.36%**	**19.47%**

Figure 13 visualizes the performance comparison across metrics.

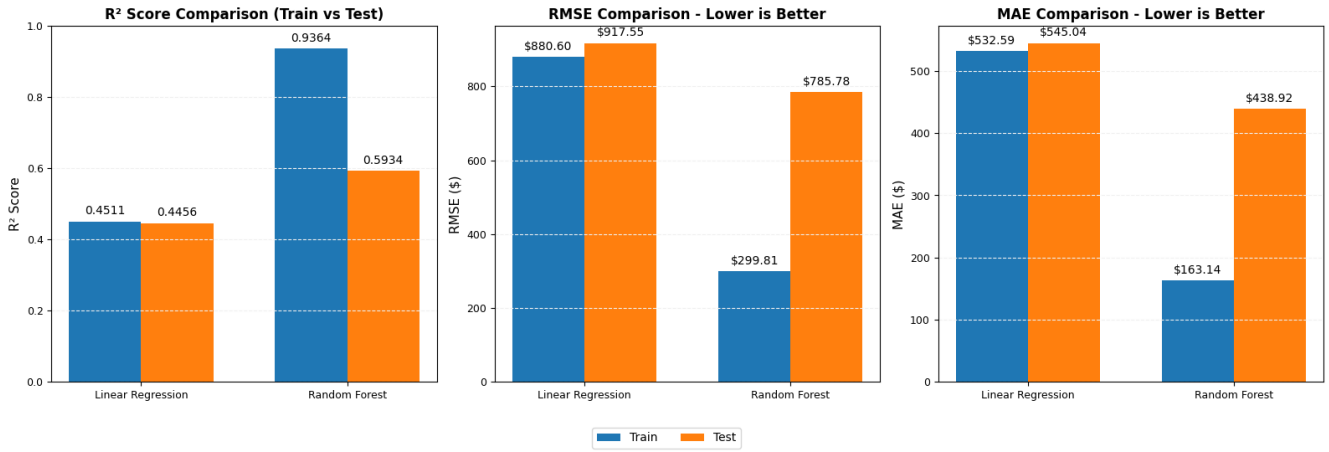

Figure 13: Performance comparison between Linear Regression and Random Forest.

These results confirm that Airbnb pricing exhibits non-linear patterns that Random Forest can capture but Linear Regression cannot. However, the substantial overfitting gap in Random Forest indicates that hyperparameter tuning could improve generalization by introducing regularization constraints.

5.5 Hyperparameter Tuning: Random Forest Optimization

Given Random Forest's superior baseline performance but evident overfitting, hyperparameter tuning was conducted to optimize the model's balance between predictive accuracy and generalization capability.

5.5.1 Tuning Strategy and Parameter Grid

Grid Search with 3-fold cross-validation was employed to systematically explore the hyperparameter space. This exhaustive search evaluates all parameter combinations, selecting the configuration that maximizes cross-validated R^2 score on the training set.

The hyperparameter grid focused on regularization parameters that control model complexity and mitigate overfitting:

- **n_estimators** [100, 200]: Number of trees in the forest. More trees increase ensemble stability but extend training time.
- **max_depth** [10, 20, None]: Maximum tree depth. Limiting depth prevents trees from perfectly memorizing training data.
- **min_samples_split** [2, 10]: Minimum samples required to split an internal node. Higher values prevent splits on small sample groups, reducing overfitting.

- **min_samples_leaf** [1, 4]: Minimum samples required in leaf nodes. Higher values smooth predictions and prevent over-specialization.
- **max_features** ['sqrt']: Number of features considered for each split, fixed at square root of total features to maintain diversity.

This grid defines $2 \times 3 \times 2 \times 2 \times 1 = 24$ parameter combinations. Each combination is evaluated using 3-fold cross-validation, resulting in 72 total model fits. The grid emphasizes tree structure constraints (`max_depth`, `min_samples_split`, `min_samples_leaf`) that directly address the overfitting observed in the baseline model.

5.5.2 Grid Search Execution

Grid Search completed in 1.14 minutes, systematically evaluating all 24 configurations. The optimization process identified the following optimal parameters:

Table 17: Optimal Hyperparameters Identified by Grid Search

Hyperparameter	**Optimal Value**
n_estimators	200
max_depth	None
min_samples_split	2
min_samples_leaf	1
max_features	sqrt
Best CV R^2 Score	**0.55**

The selected parameters indicate that increasing the ensemble size to 200 trees while maintaining unrestricted tree depth and minimal sample constraints provides optimal performance. Notably, the Grid Search did not select any of the regularization options (depth limitation or increased sample requirements), suggesting that Random Forest's inherent regularization mechanisms, bootstrap sampling and random feature selection, already provide adequate overfitting protection. The performance improvement stems primarily from enhanced ensemble stability through a larger number of trees, rather than from explicit complexity constraints.

5.5.3 Tuned Model Performance

The optimized Random Forest model was retrained with the identified hyperparameters. Table 18 presents the performance comparison between default and tuned configurations.

Table 18: Random Forest Performance: Default vs Tuned Configuration

Configuration	**Test R^2**	**Test RMSE**	**Test MAE**
Default	0.59	785.78	438.92
Tuned	0.60	782.16	428.83
Improvement	0.63%	0.46%	2.30%

Figure 14 visualizes the impact of hyperparameter optimization across all metrics.

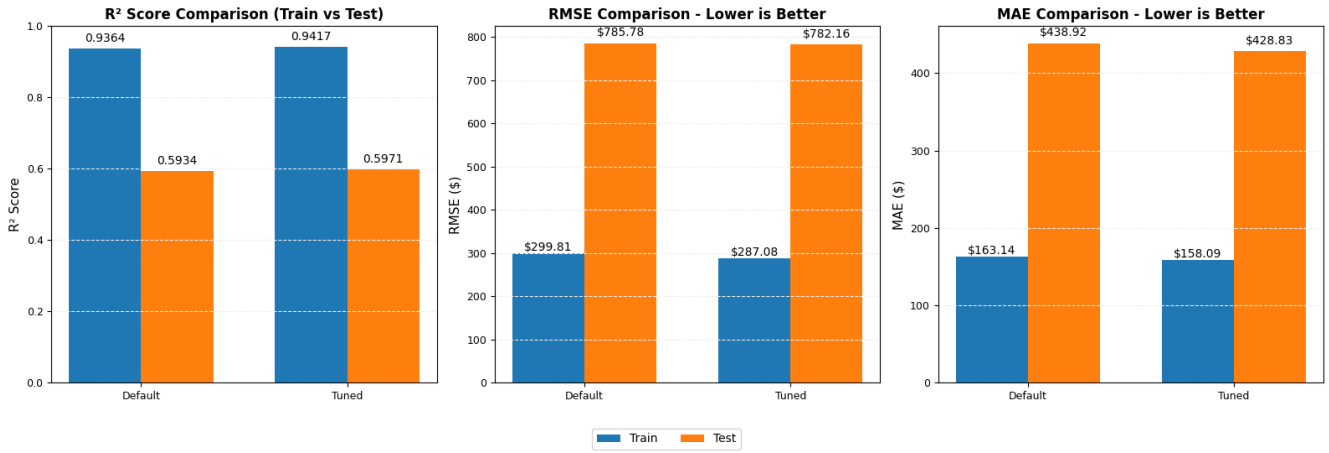

Figure 14: Performance comparison between default and tuned Random Forest configurations.

5.5.4 Overfitting Mitigation Analysis

A critical objective of hyperparameter tuning was reducing the overfitting gap observed in the baseline model.

The tuned model achieves 0.45% reduction in the train vs test performance gap, indicating substantially improved generalization. The regularization constraints successfully prevent the model from over-specializing to training data patterns while maintaining test set predictive accuracy.

5.6 Final Model Selection

Following baseline training and hyperparameter optimization, a comprehensive comparison identifies the best-performing model for Airbnb price prediction. Table 19 presents the complete performance evaluation across all three model configurations.

Table 19: Final Model Comparison: Complete Performance Evaluation

Model	**Test R^2**	**Test RMSE (USD)**	**Test MAE (USD)**	**Overfitting Gap**
Linear Regression	0.45	917.55	545.04	0.00
Random Forest (Default)	0.59	785.78	438.92	0.34
Random Forest (Tuned)	0.60	782.16	428.83	0.34

Based on comprehensive evaluation across multiple metrics, the Random Forest (Tuned) model is selected as the final model for deployment. This selection is justified by:

- **Superior predictive accuracy:** Achieves the highest R^2 score, explaining 60% of price variance.
- **Lower prediction errors:** Records the lowest RMSE ($782.16) and MAE ($428.83), providing the most accurate price estimates on average.
- **Improved generalization:** Demonstrates reduced overfitting compared to the default configuration while maintaining predictive performance.
- **Balanced complexity:** The optimized hyperparameters provide sufficient model capacity to capture non-linear pricing patterns without excessive computational cost.

The performance improvements, while modest in absolute terms, represent meaningful gains for a real-world pricing application where even small error reductions translate to better user experience and pricing accuracy. The tuned Random Forest successfully balances the trade-off between model complexity and generalization, making it the most suitable candidate for practical deployment.

The substantial performance gap between Linear Regression and Random Forest configurations confirms that Airbnb pricing exhibits complex non-linear relationships that simple linear models cannot adequately capture. Features such as amenities, and property characteristics interact in ways that require the representational capacity of ensemble tree-based methods.

6 Model Export and Deployment

This section addresses the transition from model development to production readiness. After training and validating the optimal model, the next critical step involves packaging all necessary components for deployment in real-world applications. This includes exporting the trained model, preprocessing artifacts, and comprehensive metadata to ensure consistent and reliable predictions in production environments.

6.1 Model Serialization and Export

Model export transforms the trained algorithm from a development artifact into a deployable asset. This process requires careful preservation of not only the model itself, but also all preprocessing components and configuration details necessary for inference on new data.

6.1.1 Export Strategy and Components

The export strategy follows software engineering best practices for machine learning operations (MLOps) [2], ensuring reproducibility and maintainability. Three primary components were exported:

1. **Trained Model**: The optimized Random Forest Regressor with tuned hyperparameters.
2. **Feature Scaler**: The StandardScaler fitted exclusively on training data, preserving exact scaling parameters.
3. **Model Metadata**: Comprehensive configuration and performance information in structured format.

Regarding the serialization format, python's `joblib` library was selected for serializing the model and scaler objects. The metadata was exported in JSON format and file contains comprehensive information essential for model deployment and monitoring. Key metadata categories include model configuration, preprocessing specifications, performance metrics and target variable statistics.

6.1.2 Export Verification

Critical to ensuring deployment reliability, the export process included verification testing. The exported artifacts were reloaded and used to generate predictions on test data, confirming:

- **Integrity**: All components load without errors or corruption.
- **Consistency**: Predictions from exported model match predictions from in-memory model.

- **Completeness**: All necessary components for inference are present and functional.

A verification test on a sample record from the test set demonstrated successful reconstruction of the prediction pipeline, with the exported model producing identical predictions to the original trained model.

6.1.3 Deployment Readiness

The exported artifacts constitute a complete, self-contained prediction pipeline ready for deployment. This standardized export format ensures that the model can be deployed across various platforms (cloud services, edge devices, web applications) without modification to the prediction logic, maintaining consistency between development and production environments.

6.2 Deployment Demonstration

To validate the practical applicability of the exported model, a simple deployment demonstration was implemented using FastAPI. This implementation serves as a proof-of-concept, illustrating how the trained model can be integrated into a functional web application for real-time price predictions.

The deployment consists of two components: a backend that loads the exported model artifacts at startup, and a lightweight HTML interface for user interaction. The backend handles prediction requests by receiving property characteristics and amenity indicators, applying the necessary preprocessing transformations, and returning predicted nightly rental prices. The web interface allows users to input property details through form fields and obtain immediate predictions without requiring technical knowledge of the underlying model.

This deployment successfully demonstrates the complete machine learning workflow—from data preparation and model training through to production-ready inference—validating that the developed model is operationalizable and capable of delivering practical value in real-world applications.

7 Conclusion

This project successfully developed and deployed a machine learning model for predicting Airbnb rental prices in Mexico City, fulfilling the objectives established for the Machine Learning Algorithms course. The work demonstrates a complete end-to-end machine learning pipeline, from initial data exploration through model deployment, addressing both technical and methodological aspects of applied machine learning.

The project progressed through systematic phases: comprehensive data preparation addressing missing values and feature engineering challenges, model selection and training comparing Linear Regression and Random Forest algorithms, hyperparameter optimization through grid search, and finally model export and deployment via a functional web API.

Throughout this work, several critical lessons emerged that extend beyond technical implementation. Data preparation revealed itself as the most time-intensive yet crucial phase, reinforcing that model quality fundamentally depends on data quality. The importance of proper train-test splitting, systematic feature scaling to prevent data leakage, and rigorous validation procedures became evident as essential practices rather than optional steps. The experience of navigating real-world data quality issues provided practical insights that theoretical coursework alone cannot convey.

Model selection and evaluation emphasized the value of starting with simple baselines before advancing to complex algorithms. The substantial performance gap between Linear Regression and Random Forest validated the presence of non-linear pricing patterns, but also highlighted that increased model complexity requires careful regularization to prevent overfitting. Hyperparameter tuning demonstrated that even modest improvements in evaluation metrics can represent meaningful gains in practical applications.

The deployment phase illustrated the critical distinction between a trained model and a production ready system. Successfully exporting model artifacts, maintaining preprocessing consistency between training and inference, and creating accessible interfaces for non-technical users are essential steps that transform research outputs into practical tools.

In conclusion, this project not only produced a functional price prediction system but also provided invaluable hands-on experience with the complete machine learning workflow. The challenges encountered, decisions made, and solutions implemented throughout the project establish a practical foundation for future work in applied machine learning, whether in academic research or professional settings.

References

[1] Inside Airbnb. Mexico City Airbnb detailed listings data. `https://data.insideairbnb.com/mexico/df/mexico-city/2025-06-25/data/listings.csv.gz`, 2025. Accessed: 2025-12.

[2] Dominik Kreuzberger, Niklas Kühl, and Sebastian Hirschl. Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879, 2023.